

Intelligent Customer Success & Service Automation Platform

Kowsik Y

NLBIT25014

7376232AL145

Set – 2

1. Problem Statement

A rapidly growing technology company receives customer requests through voice calls, chat, emails, and web portals. Customers ask about product features, pricing, subscriptions, troubleshooting, warranty, order status, account issues, and service requests.

The company wants to build a unified AI-powered Customer Success Platform that serves as the first point of contact for customers. The platform intelligently understands customer requests, gathers missing information through conversation, retrieves relevant organizational knowledge, determines the appropriate course of action, and completes tasks whenever possible. If sufficient information is unavailable or the request involves sensitive business operations, the system recognizes its limitations and seamlessly transfers the conversation to a human support representative.

Customers often ask follow-up questions, interrupt conversations, change topics midway, and switch between voice and text interactions. The assistant maintains conversational context while providing fast, reliable, and professional responses.

The organization expects the solution to improve customer experience without compromising security, accuracy, or business policies. The system is designed to handle real-world ambiguity, conflicting information, and incomplete customer inputs while remaining scalable for thousands of concurrent users.

Objective & Mission: To design and develop an industry-ready AI platform capable of delivering intelligent customer assistance by making appropriate architectural decisions, effectively utilizing AI technologies, and ensuring a production-quality user experience.

1.2 Project Overview & Purpose

The **Intelligent Customer Success & Service Automation Platform** is an advanced, omnichannel enterprise customer engagement solution designed to unify real-time browser-based web support, streaming conversational voice interactions, resilient business process orchestration, and human-in-the-loop escalation into a single, cohesive ecosystem. Built as a polyglot system, the platform bridges the gap between conversational artificial intelligence and deterministic enterprise workflow execution.

1.3 The Real-World Problem Answered

Modern enterprise customer service operations frequently suffer from organizational and technological fragmentation. Traditional chat systems rely on rigid decision trees that frustrate users, while standalone LLM conversational agents lack secure, transactional connectivity to core business databases and workflow systems. Furthermore, integrating low-latency conversational audio streaming alongside asynchronous human agent intervention presents severe concurrency, state consistency, and security challenges.

This platform solves these systemic friction points by implementing an architecture where conversational AI handles fluid natural language reasoning and retrieval, while deterministic workflow engines manage business state transactions, retry logic, and compliance approvals.

1.4 Target Audience Breakdown & Benefits

- **For Non-Technical Stakeholders & Business Leaders:**

The system automates high-volume customer inquiries across text and voice channels without human intervention, drastically lowering support operating costs. When complex requests arise (e.g., high-value warranty replacements or refunds exceeding policy limits), the platform automatically escalates to a human support agent with complete conversational context, sentiment evaluation, and fraud risk scoring, ensuring zero customer repeat-explanation and guaranteed policy compliance.

- **For Software Engineers & Solution Architects:**

The project implements a well-bounded polyglot architecture utilizing TypeScript (Next.js/NestJS) for I/O-bound web traffic and API gateways, Python for compute-heavy AI/ML audio processing and LLM graph orchestration, and Temporal for resilient distributed workflows. Clean schema isolation via PostgreSQL domain namespaces, OpenTelemetry distributed tracing, and specialized security shields ensure extensibility, testability, and operational safety.

2. Architectural Style & Core Design Decisions

2.1 Primary Architecture Style: Polyglot Modular Monolith & Specialized Microservices

The system deliberately eschews both monolithic entanglement and premature microservice sprawl. Derived from exhaustive engineering analyses documented in our Architectural Decision Records (ADRs), the platform adopts a **Polyglot Hybrid Architecture**:

1. **Modular Domain Monolith for Core Business Logic:**

The primary data-processing application is implemented as a modular core service built on NestJS and Fastify. Rather than deploying dozens of separate chat, case, and policy microservices—which introduce network overhead and complex distributed transactions—the core domain is cleanly bounded by logical schema separation inside PostgreSQL (`customer` , `conversation` , `case_management` , `policy` , `audit`).

2. **Specialized Domain Microservices for Heterogeneous Workloads:**

Services exhibiting divergent computational requirements, scaling metrics, or linguistic ecosystems

are completely decoupled into dedicated microservices communicating over standardized interfaces:

- **AI Orchestrator Service:** Built in Python 3.12 with FastAPI and LangGraph to leverage the dominant AI/ML dependency ecosystem and handle non-linear stateful conversational graphs.
- **Voice Service:** Built in Python to manage persistent bi-directional binary WebSocket streams (`pcm_s16le` at 16kHz) and real-time inference (Faster-Whisper STT and TTS synthesis) without blocking general web gateway worker threads.
- **Durable Workflow Worker:** Built on the Temporal SDK to handle long-running, asynchronous background processes (e.g., multi-day refund review approvals) that survive service restarts and system failures without losing transaction state.

2.2 Summary of Foundational Architectural Decision Records (ADRs)

ADR Identifier	Engineering Decision	Rationale & Enterprise Impact
ADR 0001	Next.js as BFF, Not Sole Gateway	Utilizes Next.js Route Handlers strictly as a Backend-for-Frontend (BFF) session coordination layer for browser clients, while maintaining independent microservice port routing and API gateways for uniform external rate limiting, authentication, and partner hooks.
ADR 0002	LangGraph for AI Flow, Temporal for Workflows	Separates probabilistic AI conversation logic from deterministic business transactions. LangGraph manages dialogue state, retrieval evaluation, and tool proposals; Temporal enforces retry policies, database commits, timers, and supervisor approvals.
ADR 0003	Binary PCM WebSocket for Voice MVP	Leverages browser <code>AudioWorklet</code> streaming via raw PCM binary frames over WebSockets rather than heavyweight telephony protocols, achieving minimal latency, simple debugging, and direct interfacing with audio processing engines.
ADR 0004	Modular Core Before Microservice Splitting	Consolidates related enterprise entity management within a cohesive NestJS Core API, preventing distributed transaction complexity during initial scaling while preserving explicit database interfaces for future service extraction.

3. Overall System Architecture & Topology

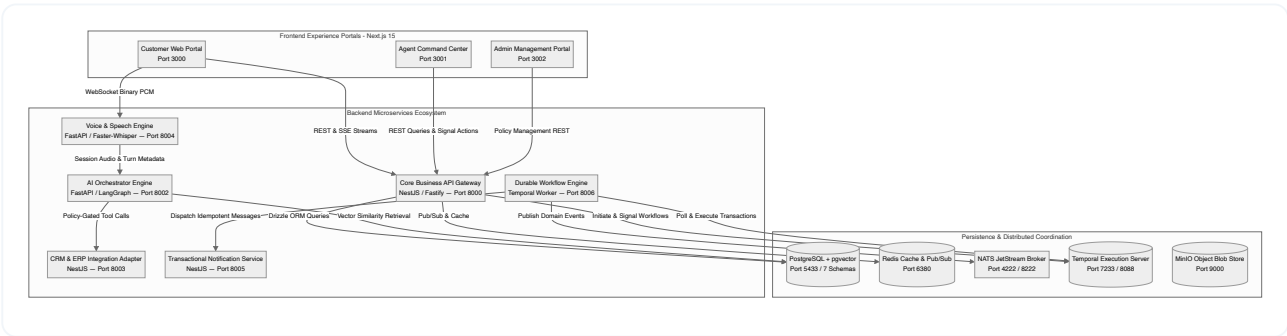


Diagram Explanation: The architecture separates browser user interface portals from microservice execution gateways and persistent backend databases. Client applications interact either via REST and HTTP Server-Sent Events for chat, or persistent WebSockets for live voice audio. Business state is secured within PostgreSQL across isolated schemas, while long-running operations are routed through Temporal and NATS JetStream.

4. Repository & Module Organization

The project is governed as a unified polyglot monorepo utilizing **pnpm workspaces**, **Turborepo** task orchestration, and isolated Python virtual environments (`.venv`). This enforces unified architectural linting, unified continuous integration testing, and reusable code libraries without publishing artifacts to external package registries.

├─ apps/	# Next.js 15 Web Portals (Frontend Layer)
│ └─ customer-web/	# Client interface for live web support chat & microphone audio
│ └─ agent-console/	# Real-time human support agent triage, supervisor approvals
│ └─ admin-console/	# Platform governance, LLM model configuration, and policy management
├─ services/	# Autonomous Domain & Computational Microservices
│ └─ core-api/	# Primary NestJS/Fastify domain gateway and PostgreSQL database
│ └─ ai-orchestrator/	# Python FastAPI service executing stateful LangGraph conversational agents
│ └─ voice-service/	# Python bi-directional audio WebSocket server with VAD, STT, and TTS
│ └─ workflow-worker/	# Temporal TypeScript worker executing resilient, multi-step business processes
│ └─ integration-service/	# Canonical NestJS adapter unifying external CRM, ERP, and other systems
│ └─ notification-service/	# Idempotent transactional email and SMS messaging gateway
│ └─ knowledge-service/	# Specialized retrieval microservice scheduled for enterprise RAG
├─ packages/	# Shared TypeScript Enterprise Modules
│ └─ contracts/	# Shared Zod schemas, data transmission objects (DTOs), and constants
│ └─ events/	# Standardized event definitions and NATS pub/sub message contracts
│ └─ auth/	# Distributed JWT verification, session management, and RBAC logic
│ └─ observability/	# OpenTelemetry correlation propagation, Prometheus metric exporters
│ └─ ui/	# Responsive design system tokens, glassmorphic UI components
├─ python-packages/	# Shared Python Domain Libraries
│ └─ ai-contracts/	# Pydantic serialization schemas matching TypeScript contracts
│ └─ ai-observability/	# Python OpenTelemetry tracers and specialized prompt/response logging
│ └─ service-auth/	# Inter-service authentication validation and secure API key management
│ └─ rag-core/	# pgvector indexing utilities, chunking algorithms, and embedding management
└─ infrastructure/	# Container infrastructure manifests, database initialization scripts

5. End-to-End System Workflow & Request Lifecycle

When a customer initiates an interaction on the platform, information moves through a well-defined sequence of architectural boundaries. The lifecycle differs significantly between text-based chat inquiries and live voice conversations.

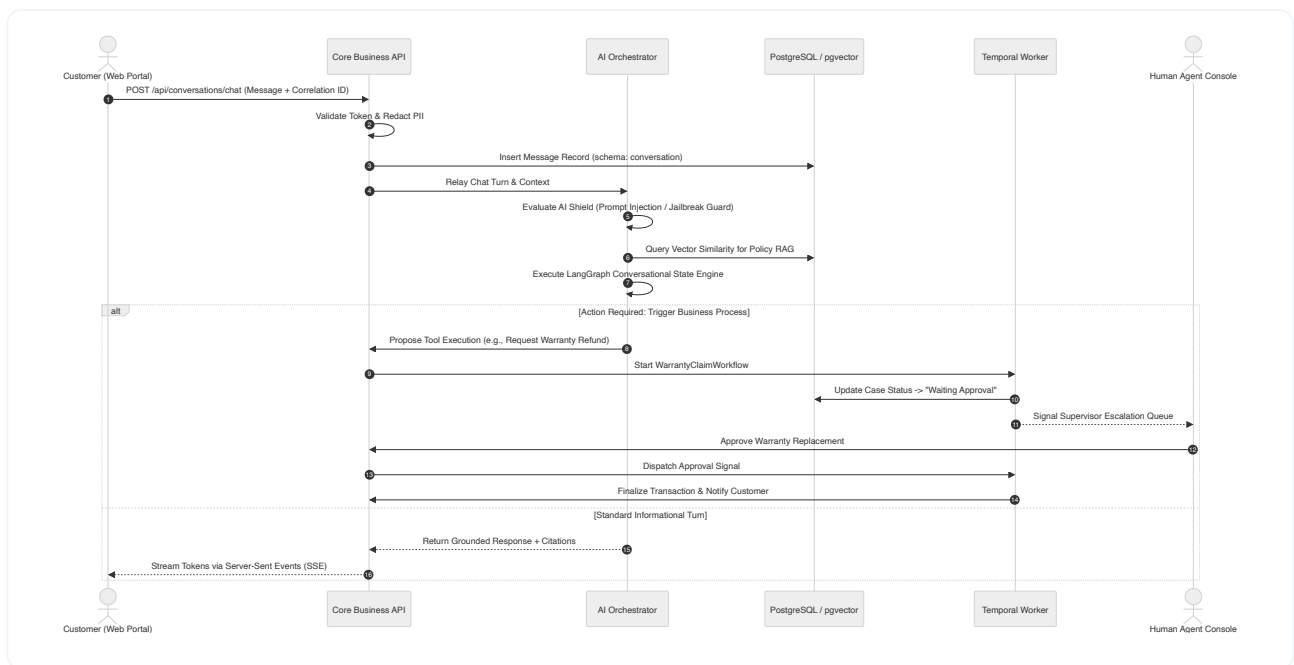


Diagram Explanation: The sequence demonstrates the transactional boundary between probabilistic chat reasoning and deterministic business actions. Notice that the AI Orchestrator never mutates production database rows or issues financial refunds directly; instead, it generates structured tool proposals that are validated by the Core API and reliably executed via Temporal workflows.

5.1 Step-by-Step Request Lifecycle Breakdown

1. Edge Ingestion & Validation:

Requests from browser applications arrive at the Next.js BFF and are routed to the Core API. Requests undergo immediate bearer token authentication, payload schema validation via Zod, and correlation ID assignment for distributed observability.

2. Security Scrubbing & Pre-Processing:

The logging interceptor evaluates payloads against regex-based PII redaction rules, instantly masking credit card figures, social security numbers, and bank details from telemetry streams.

3. Dialogue Reasoning & Intent Classification:

The chat turn is dispatched to the AI Orchestrator. The engine checks the prompt against an adversarial attack filter (AI Security Shield). Once verified safe, intent classifiers determine whether the customer seeks simple answers, order status updates, or complex warranty dispute interventions.

4. Knowledge Grounding (RAG):

For informational queries, vector embeddings are generated and compared against pre-computed enterprise policy documents stored in PostgreSQL **pgvector**. Matched passages are supplied as strict domain context to the inference engine to eliminate hallucination.

5. Deterministic Process Hand-off (When Applicable):

If the conversation necessitates mutating enterprise state (such as initiating a product return), LangGraph suspends the dialog loop and submits a validated invocation payload to the Core API. The Core API initiates a Temporal durable workflow, guaranteeing execution even in the event of temporary infrastructure failures.

6. Asynchronous Response Delivery:

Responses return to the browser client asynchronously. For chat interfaces, textual responses

stream character-by-character over HTTP Server-Sent Events (SSE). For voice sessions, generated text is converted to streaming PCM frames delivered directly over active WebSockets.

6. Service Communication & Interaction Model

The platform utilizes a structured hybrid communication strategy, selecting specific network transport protocols tailored to operational elasticity and latency tolerances.

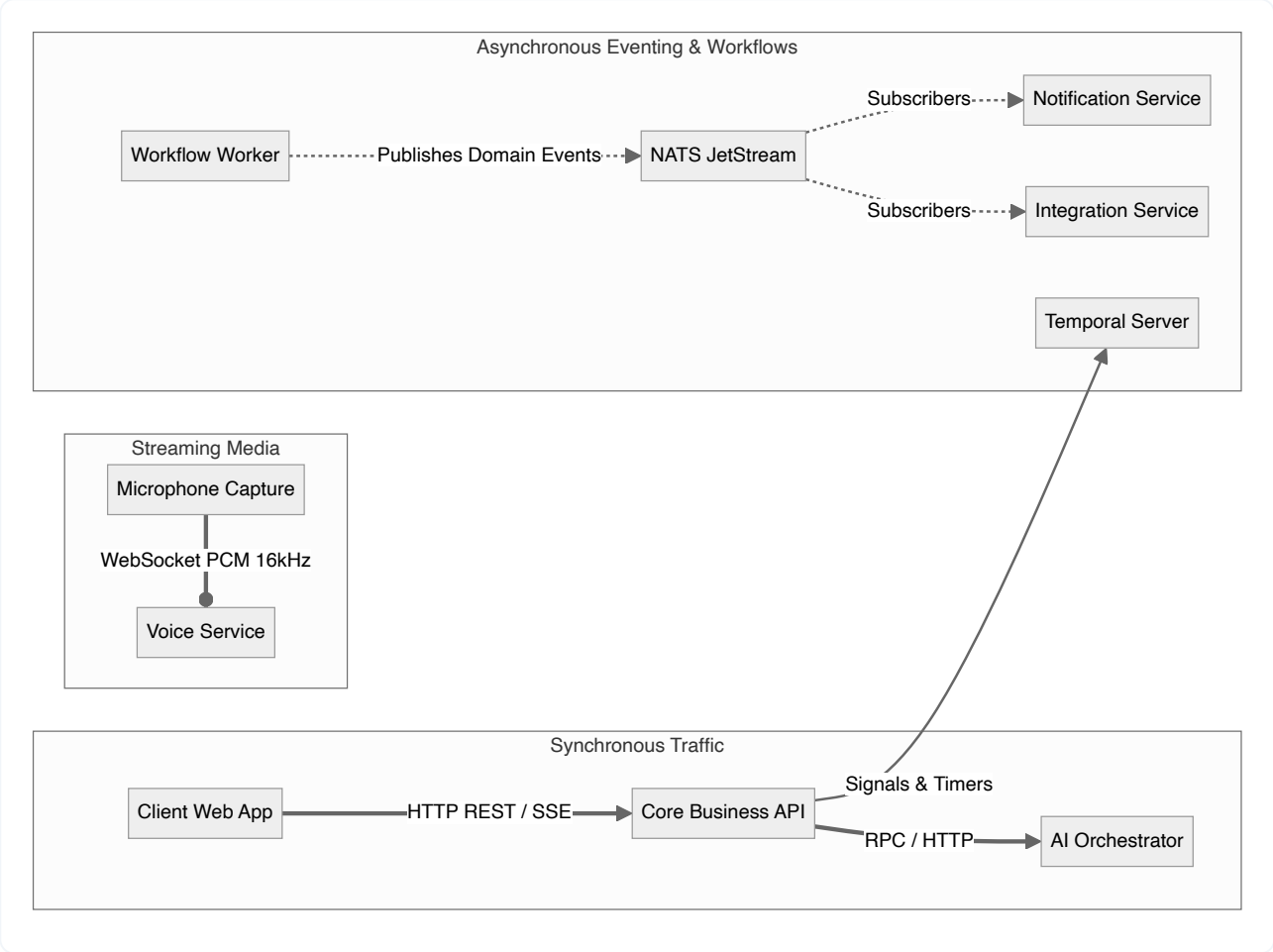


Diagram Explanation: Communication paradigms are partitioned by operational requirements. Frontends use REST and SSE for standard API interactions, raw binary WebSockets for continuous voice audio streaming, and decoupled asynchronous messaging via NATS and Temporal for resilient background execution.

6.1 Inter-Service Communication Matrix

Transport Mechanism	Primary Consumers	Architectural Rationale
HTTP REST & Server-Sent Events (SSE)	Customer Portal ↔ Core API, Core API ↔ AI Orchestrator	Delivers low-overhead REST CRUD operations and progressive streaming text generation without open-ended bi-directional websocket memory consumption.

Transport Mechanism	Primary Consumers	Architectural Rationale
Binary WebSocket (pcm_s16le , 16kHz)	Browser AudioWorklet ↔ Voice Service	Enables low-latency, full-duplex transmission of uncompressed audio frames, vital for real-time speech-to-text transcription and instant barge-in interruption.
NATS JetStream (Async Pub/Sub)	Workflow Worker, Notification Service, Integration Service	De-couples high-volume background domain events (OrderShipped , RefundApproved) with guaranteed at-least-once message delivery and persistent event replication.
Temporal SDK RPC Protocols	Core API ↔ Temporal Server ↔ Workflow Workers	Enforces strict saga management and distributed state machines with persistent execution logs and automatic disaster recovery retries.

7. Data Flow & Storage Architecture

The platform avoids unified monolithic database tables in favor of strict domain partitioning within PostgreSQL, supplemented by rapid in-memory key-value caching and scalable binary object storage.

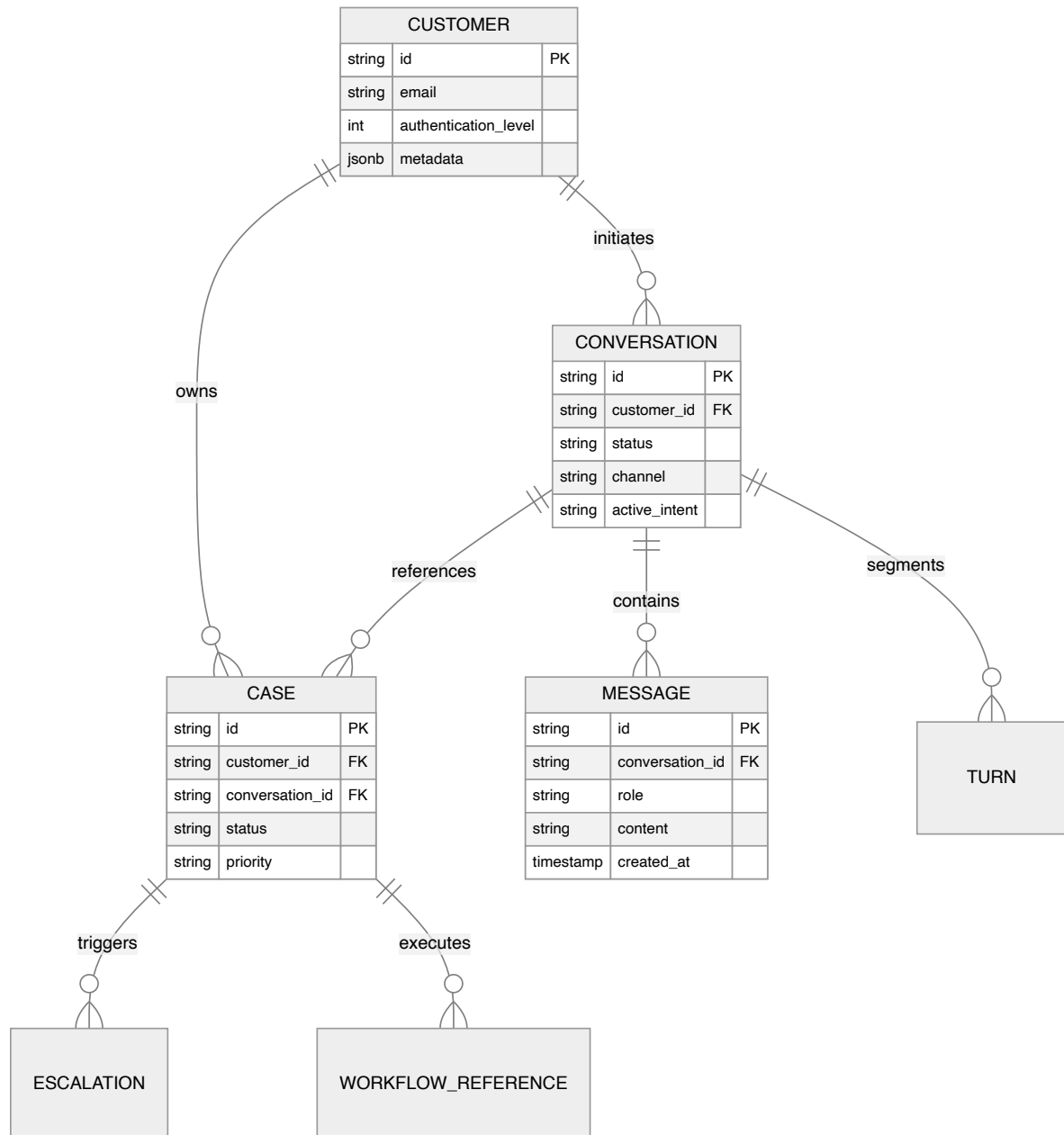


Diagram Explanation: Entity relationships highlight strict foreign-key integrity between customer identities, conversational dialogue records, and durable case tickets. Drizzle ORM enforces these relational contracts across bounded schema namespaces.

7.1 PostgreSQL Schema Namespaces

The primary database instance is structured into seven isolated relational namespaces to prevent unauthorized cross-domain data mutations and facilitate future microservice decomposition:

- **customer Schema:** Manages core end-user identities, authentication levels, localized preferences, and custom profile metadata.
- **conversation Schema:** Records interactive sessions, storing individual message turns, speaker roles (user, agent, assistant), sentiment tags, and intent labels.
- **case_management Schema:** Maintains support tickets, escalation trigger justifications, priority levels, and human agent task assignments.

- **policy Schema:** Contains organizational rules, product warranty constraints, and multi-dimensional vector embeddings for conversational retrieval.
 - **audit Schema:** Functions as an append-only cryptographic transaction log tracking administrative overrides, AI tool execution proposals, and policy mutations.
 - **workflow_reference Schema:** Binds Temporal workflow execution identifiers to business case entities for seamless UI state tracking.
 - **notification Schema:** Logs outgoing transactional communication delivery states, retry counts, and customer interaction receipts.
-

8. Business Logic & Durable Workflow Automation

A foundational engineering pillar of this project is the delegation of multi-step business transactions to **Temporal**, ensuring resilient state progression that outlives application crashes, networking failures, or external service downtime.

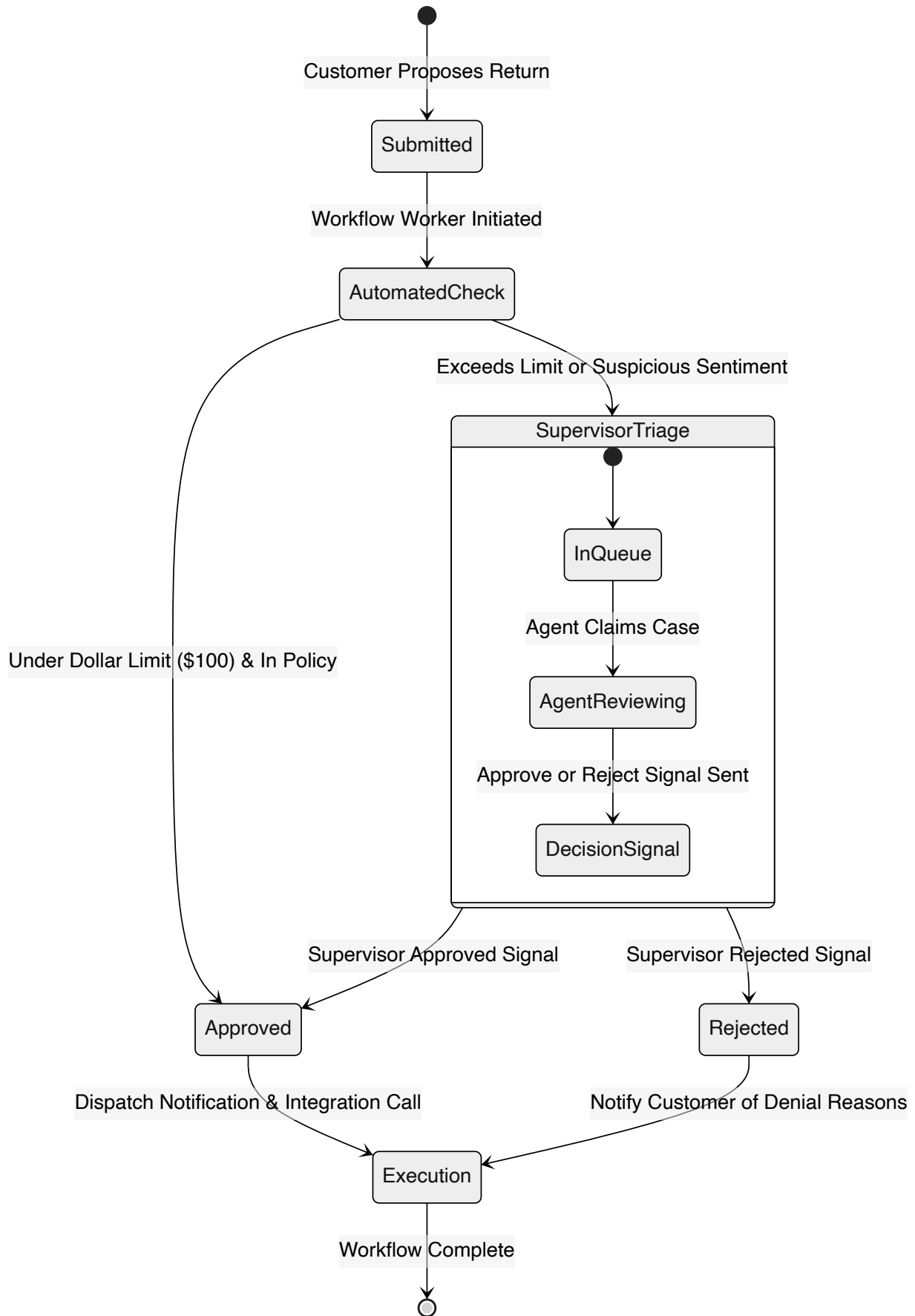


Diagram Explanation: The state machine illustrates the resilient execution path of a Warranty Claim Workflow. If business validation requires human approval, Temporal suspends worker thread execution—consuming zero active CPU resources—and sets a durable timer while waiting for an external signal from the human Agent Console.

8.1 Implemented Resilient Workflows

1. Warranty Claim Workflow (`WarrantyClaimWorkflow`):

Evaluates customer return requests against hardware policy timelines. If the claimed item value falls below an automated confidence threshold (\$100) and invoice verification succeeds via the Integration Service, the workflow instantly authorizes replacement shipping and notifies the customer. If validation criteria fail, it generates an escalation entity in the `case_management` schema and pauses execution pending human agent review.

2. Refund Review Workflow (`RefundReviewWorkflow`):

Orchestrates high-value financial refunds. Enforces a strict Human-in-the-Loop requirement, requiring an agent to execute an explicit cryptographic approval signal via the Agent Console before transactional settlement commands are dispatched to integration adapters.

3. Order Processing Workflow (`OrderProcessingWorkflow`):

Monitors shipment tracking data, handling automated notification dispatch when delivery dates shift or logistics carriers report fulfillment exceptions.

9. AI & Real-Time Voice Streaming Architecture

9.1 AI Orchestrator & Conversational State Machine (LangGraph)

The AI engine transforms conversational turns from stateless, reactive completion calls into structured, multi-step dialogue graphs capable of handling complex customer service operations.

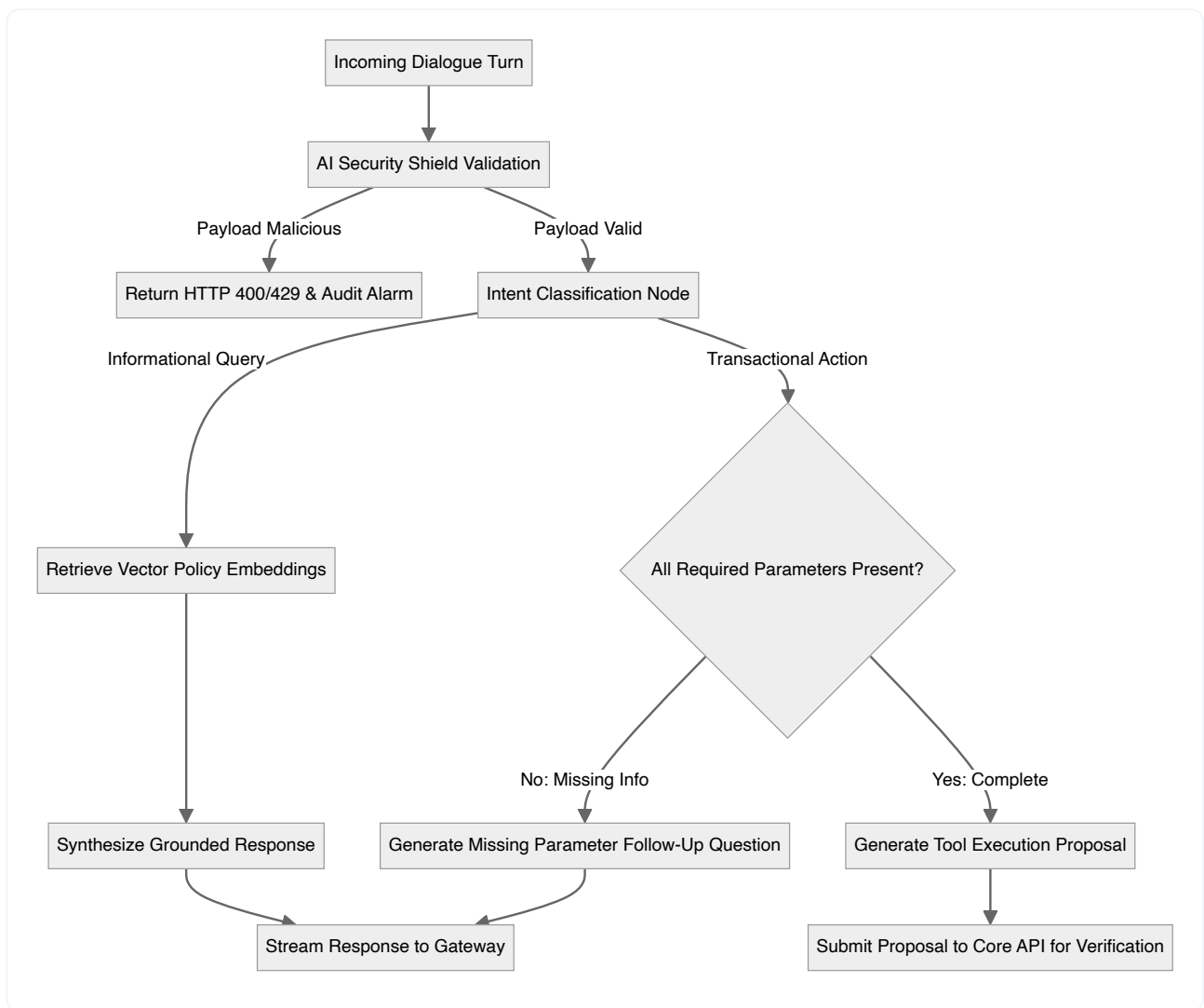


Diagram Explanation: The LangGraph processing sequence ensures safety and parameter completeness. Before invoking external enterprise tools, the engine evaluates whether required arguments (e.g., Order Number, Serial ID) are present; if incomplete, it loops back to interrogate the user without executing premature backend transactions.

9.2 Real-Time Voice Streaming Engine (Voice Service)

The platform delivers seamless voice conversations without relying on legacy telephony hardware by implementing a bi-directional streaming pipeline built on Python and continuous web sockets.

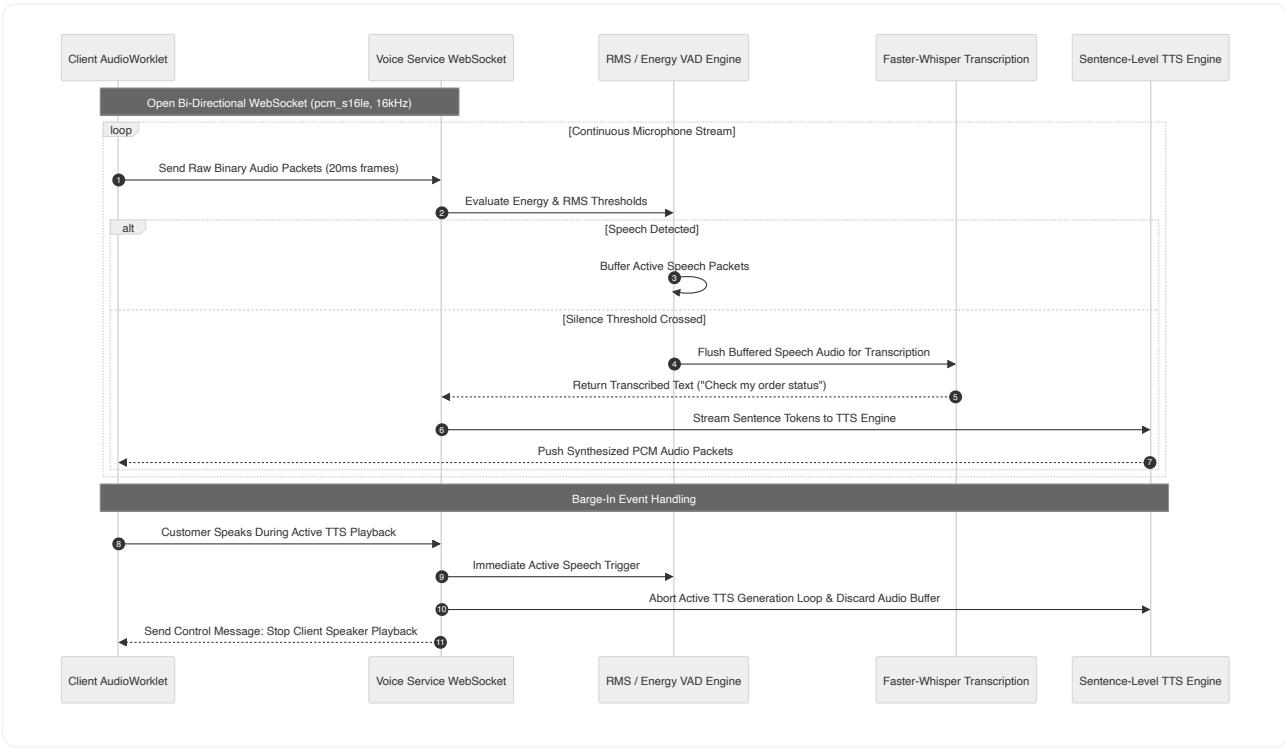


Diagram Explanation: The voice engine processes continuous 16kHz audio streams in real time. When Voice Activity Detection (VAD) identifies active customer speech during audio response playback, the server instigates an immediate "barge-in" interruption, truncating TTS output and ensuring rapid conversational responsiveness.

10. API Gateway & Microservice Endpoint Reference

The system surfaces clean, domain-driven API boundaries. Below is an exhaustive structural table characterizing the major network ingress interfaces across active platform services:

Service Boundary	Port	Endpoint Path	Method	Authentication & Validation	Architectural Responsibility
Core Business API	8000	/api/v1/conversations/chat	POST	Bearer Token, Zod DTO Schema	Receives chat messages, executes PII masking, inserts database conversation turns, and relays context to AI Orchestrator.
Core Business API	8000	/api/v1/cases/:id/escalation	GET / POST	Agent RBAC Token, Zod Schema	Retrieves escalated customer cases or initiates supervisory workflow signal override events.

Service Boundary	Port	Endpoint Path	Method	Authentication & Validation	Architectural Responsibility
AI Orchestrator	8002	/v1/orchestration/turn	POST	Service-to-Service Secret, Pydantic	Executes stateful LangGraph conversation transitions, intent matching, RAG document evaluation, and tool proposals.
Voice Service	8004	/v1/voice/stream	WS	Session JWT Handshake over HTTP Upgrade	Opens raw bi-directional websocket connection for uncompressed 16kHz PCM audio streaming, VAD, STT, and TTS.
Integration Adapter	8003	/api/v1/adapters/orders/:id	GET / PUT	Service Secret, Zod Validation	Retrieves canonical fulfillment data or dispatches enterprise ERP ledger modification commands.
Notification Engine	8005	/api/v1/dispatch/template	POST	NATS Internal Messaging Contract	Triggers idempotent customer communications (email/SMS) with automated failure retry policies.
Knowledge Service	8001	/health/live / ready	GET	Open Telemetry Probes	Exposes Kubernetes liveness and database readiness probes for vector indexing deployment monitoring.

11. Security, Privacy & Observability Shield

Security and telemetry are woven directly into inter-service middleware rather than added as an afterthought.

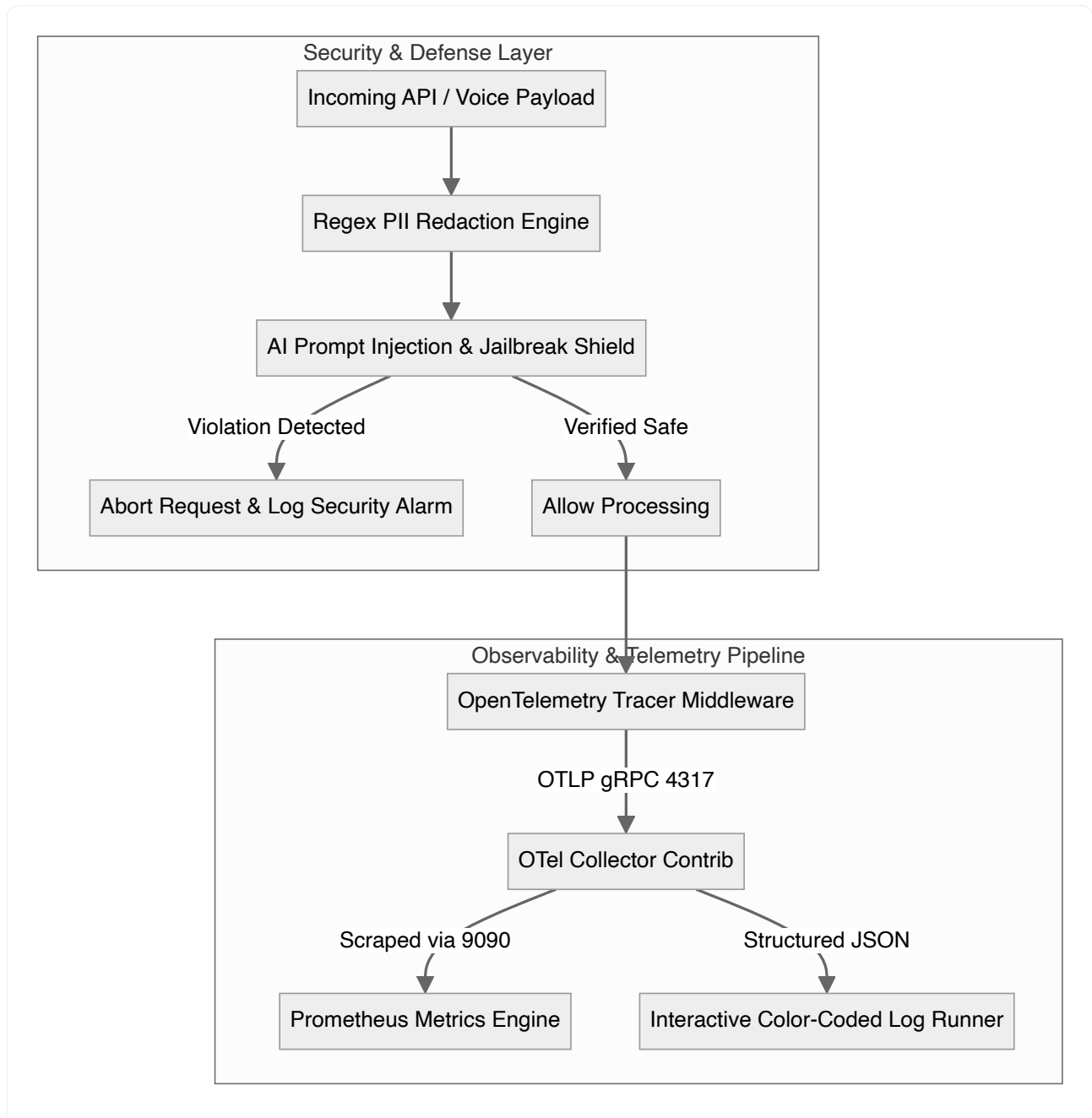


Diagram Explanation: Security interception occurs prior to execution logic. Payloads are cleansed of personally identifiable information and scrutinized for AI attacks before execution spans are serialized by OpenTelemetry and exported to Prometheus monitoring instances.

11.1 Implemented Defense Mechanisms

- **AI Security Shield:**

Located inside the AI Orchestrator middleware (`security.py`), this subsystem intercepts every text conversation turn and transcribed voice input. It uses heuristic analysis and boundary semantics to identify adversarial prompt injection, jailbreak attempts, and instruction override attacks, terminating malicious sessions instantly with protective HTTP 400/429 status codes.

- **Automated PII Redaction Engine:**
Configured across shared Node.js (`packages/observability`) and Python (`python-packages/ai-observability`) libraries, explicit regex parsers scan all outbound application log strings and telemetry attributes, masking Social Security numbers, credit card sequences, and sensitive banking data to ensure HIPAA/SOC-2 compliance.
- **Distributed DDoS Rate Limiters & Correlation Tracing:**
Sliding-window algorithms limit ingress request volume across external routing gates. Every inbound request receives a globally unique `X-Correlation-ID` header that persists across synchronous REST boundaries, NATS message headers, and Temporal workflow payloads, guaranteeing unified distributed tracing.

12. External Third-Party Integrations

The repository relies on specific enterprise platform integrations to achieve inference capabilities and business automation:

- **OpenAI & Compatible LLM Inference APIs:**
The AI Orchestrator integrates with remote LLM completion engines using standardized messaging protocols, referencing advanced enterprise models (`gemini-2.5-flash` , `gpt-4o-mini-tts` , and `gpt-4.1-nano`) for rapid intent classification, textual generation, and sentence synthesis.
- **Temporal Cloud / Distributed Execution Engine:**
Establishes durable gRPC connections to local Temporal containers (or managed cloud clusters) to serialize transactional workflow history and orchestrate fault-tolerant worker execution schedules.
- **Canonical Enterprise CRM & ERP Adapters:**
The Integration Service implements canonical data transformation contracts designed to communicate with external shipping logistics providers and merchant order databases without coupling internal schemas to vendor-specific APIs.

13. Categorized Technology Stack

Architecture Domain	Technology Implementations & Specifications
Frontend Web Layer	Next.js 15 (App Router), React 19, TypeScript 5.7, Tailwind CSS, Glassmorphism UI Components, Browser <code>AudioWorklet</code>
Backend API Gateways	NestJS 10, Fastify High-Performance HTTP Adapter, Zod Input Validator, OpenAPI / Swagger Automated Docs
AI & Conversational Core	Python 3.12, FastAPI, LangGraph, LangChain, Pydantic V2, OpenAI / Gemini Compatible Completion APIs
Real-Time Voice Engine	Python FastAPI WebSockets, Faster-Whisper Local STT Transcription Engine, Real-Time Sentence TTS, RMS Energy VAD

Architecture Domain	Technology Implementations & Specifications
Database & Blob Storage	PostgreSQL 17, <code>pgvector</code> Extension for Semantic Embedding Search, Drizzle ORM, MinIO Object Storage
Async & Resilient Engines	Temporal SDK Worker Engine (Resilient Business Workflows), Redis 7 (Caching), NATS JetStream (Message Broker)
Telemetry & Observability	OpenTelemetry gRPC / HTTP Exporters, Prometheus Telemetry Scraping, Regex PII Scrubbers
Monorepo Infrastructure	pnpm 9 Workspaces, Turborepo 2.10.8 Task Orchestration, Docker & Docker Compose, Automated Multi-Service Shell Runner (<code>run_all_services.sh</code>)

14. Verification & Operational Testing Standards

To maintain structural reliability across both TypeScript and Python language domains, the project imposes strict continuous integration testing standards governed by automated monorepo execution targets:

1. TypeScript Comprehensive Quality Gates:

Executed across root, applications, and Node services via Turborepo:

```
pnpm test && pnpm build && pnpm typecheck && pnpm lint
```

Verifies Drizzle ORM structural schemas, frontend React component compilation, and contract validation rules.

2. Python AI Orchestrator Suite:

Verified within isolated virtual environments using custom Python paths:

```
cd services/ai-orchestrator && PYTHONPATH="src:../../python-packages/ai-contracts/src"
pytest tests -v
```

Tests LangGraph node transitions, intent classifier accuracy, and AI Security Shield deflection thresholds.

3. Python Voice Engine Acoustic Verification:

Tested against real-time media streaming constraints:

```
cd services/voice-service && PYTHONPATH="src:../../python-packages/ai-contracts/src"
pytest tests -v
```

Validates frame packet buffering, voice activity RMS triggers, and TTS sentence streaming timing.